

QUESTION 1

Title

Constraint-Based Problem Solving Using Backtracking Search

Problem Statement

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) while satisfying the following constraints:

Constraints

1. D1 cannot work Night shift.
2. D2 must work before D3 (Morning < Afternoon < Night).
3. Only one doctor can be assigned to each shift.
4. D3 cannot work Morning shift.
5. Every doctor must be assigned exactly one shift.

Objective

Apply the Backtracking Search Algorithm to assign doctors to shifts while satisfying all constraints. Show all intermediate constraint checks and determine the final valid schedule.

Python Code:

```
doctors = input("Enter doctors: ").split()
```

```
shifts = input("Enter shifts: ").split()
```

```
def solve(assign, i):
```

```
    if i == len(doctors):
```

```
        print("\nFinal Schedule:")
```

```
        for d in doctors:
```

```
            print(d, "->", assign[d])
```

```
        return True
```

```
    d = doctors[i]
```

```
for s in shifts:
```

```
    if s not in assign.values():
```

```
        if d == "D1" and s == "Night":
```

```
            continue
```

```
        if d == "D3" and s == "Morning":
```

```
            continue
```

```
    assign[d] = s
```

```
    if "D2" in assign and "D3" in assign:
```

```
        if shifts.index(assign["D2"]) >= shifts.index(assign["D3"]):
```

```
            del assign[d]
```

```
            continue
```

```
    if solve(assign, i + 1):
```

```
        return True
```

```
    del assign[d]
```

```
return False
```

```
solve({}, 0)
```

Output:

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\New\AppData\Local\Programs\Python\Python313\Python_Code.py =
Enter doctors: D1 D2 D3
Enter shifts: Morning Afternoon Night

Final Schedule:
D1 -> Morning
D2 -> Afternoon
D3 -> Night
>>>
```

Solution:

Step 1

Assign **D1 → Morning**

Constraint Check:

- D1 is not on Night ✓
- Shift available ✓

Accepted

Step 2

Assign **D2 → Morning**

Morning already occupied **X**

Rejected

Try Afternoon

Accepted ✓

Step 3

Assign **D3 → Morning**

D3 cannot work Morning **X**

Rejected

Try Afternoon

Already occupied **X**

Rejected

Try Night

Accepted ✓

Constraint Verification

Constraint	Status
D1 not Night	✓
D2 before D3	✓
One doctor per shift	✓
D3 not Morning	✓
One shift per doctor	✓

Final Schedule

Doctor Shift

D1	Morning
D2	Afternoon
D3	Night

Q2: Title

Robot Path Planning using Breadth-First Search (BFS)

Problem Statement

A robot operates in a 5×5 grid environment. The robot starts from S (Start) and must reach G (Goal) while avoiding obstacles.

Constraints

1. Movement allowed: Up, Down, Left, Right
2. Cost of each move = 1
3. Robot cannot move through obstacles.
4. Use Manhattan Distance Heuristic.
5. Apply Breadth-First Search (BFS) to determine the optimal path.
6. Show step-by-step node expansion and queue updates.

Grid

S O X O O

O O X O O

O O O O X

X X O X O

O O O O G

S = Start

G = Goal

X = Obstacle

O = Free Cell

Python Code:

```
from collections import deque
```

```
r, c = 5, 5
```

```
print("Enter the 5x5 grid (S, G, O, X):")
```

```
grid = [input().split() for _ in range(r)]
```

```
for i in range(r):
```

```
    for j in range(c):
```

```
        if grid[i][j] == 'S':
```

```
            start = (i, j)
```

```
        if grid[i][j] == 'G':
```

```
            goal = (i, j)
```

```
move = [(-1,0),(1,0),(0,-1),(0,1)]
```

```
q = deque([(start, [start])])
```

```
visited = {start}
```

```

while q:
    (x, y), path = q.popleft()

    if (x, y) == goal:
        print("Path:", path)
        print("Cost:", len(path)-1)
        break

    for dx, dy in move:
        nx, ny = x + dx, y + dy

        if 0 <= nx < r and 0 <= ny < c:
            if grid[nx][ny] != 'X' and (nx, ny) not in visited:
                visited.add((nx, ny))
                q.append(((nx, ny), path + [(nx, ny)]))

```

Output:

```

Enter the 5x5 grid (S, G, 0, X):
S 0 X 0 0
0 0 X 0 0
0 0 0 0 X
X X 0 X 0
0 0 0 0 G
Path: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (4, 2), (4, 3), (4, 4)]
Cost: 8

```

Solution:

. Solution

Step 1: Start

- Start at **S (0,0)**.
- Add the start node to the queue.

Queue: [(0,0)]

Step 2: BFS Expansion

- Remove the first node from the queue.

- Visit all valid neighboring cells (Up, Down, Left, Right).
- Ignore obstacles (X) and visited cells.
- Add valid neighbors to the queue.

Step 3: Continue Search

- Repeat the process until the Goal (G) is reached.

Optimal Path

(0,0)

→ (1,0)

→ (2,0)

→ (2,1)

→ (2,2)

→ (3,2)

→ (4,2)

→ (4,3)

→ (4,4)

Total Cost

- Number of moves = **8**
- Cost per move = **1**

Total Cost = 8

Manhattan Distance Formula

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

Q3. Title

Autonomous Rescue Robot using Uniform Cost Search

Aim

To design an intelligent rescue robot that safely reaches a survivor while minimizing the total travel cost using Uniform Cost Search.

Algorithm

1. Initialize the robot at the Start node.

2. Insert the start node into the priority queue.
3. Expand the node with the minimum accumulated cost.
4. Skip blocked cells.
5. Add movement cost and risk cost when applicable.
6. Continue until the Goal node is reached.
7. Display the optimal path and total cost.

Constraint Analysis

- Movement only in four directions.
- Obstacles cannot be crossed.
- Limited sensing requires online updates.
- Risk zones increase the path cost.
- The robot must minimize the total cost.

Python Code:

```
import heapq
```

```
r, c = 5, 5
```

```
print("Enter the 5x5 grid (S, G, 0, X, R):")
```

```
grid = [input().split() for _ in range(r)]
```

```
for i in range(r):
```

```
    for j in range(c):
```

```
        if grid[i][j] == 'S':
```

```
            start = (i, j)
```

```
        if grid[i][j] == 'G':
```

```
            goal = (i, j)
```

```
move = [(-1,0),(1,0),(0,-1),(0,1)]
```



```
pq = [(0, start)]
```

```
visited = set()
```

```
while pq:
```

```
    cost, (x, y) = heapq.heappop(pq)
```

```
    if (x, y) in visited:
```

```
        continue
```

```
    visited.add((x, y))
```

```
    if (x, y) == goal:
```

```
        print("Minimum Cost =", cost)
```

```
        break
```

```
    for dx, dy in move:
```

```
        nx, ny = x + dx, y + dy
```

```
    if 0 <= nx < r and 0 <= ny < c:
```

```
        if grid[nx][ny] != 'X' and (nx, ny) not in visited:
```

```
            extra = 2 if grid[nx][ny] == 'R' else 0
```

```
            heapq.heappush(pq, (cost + 1 + extra, (nx, ny)))
```

Output:

```
Enter the 5x5 grid (S, G, 0, X, R):  
S 0 R X 0  
0 X 0 0 0  
0 0 X R 0  
R 0 0 X 0  
0 0 0 0 G  
Minimum Cost = 8
```

Solution:

(a) Constraint Analysis

- The robot moves only Up, Down, Left, and Right.
- It cannot pass through obstacles (X).
- Each move costs 1.
- Entering a Risk Zone (R) adds 2 extra cost.
- The robot updates its path based on nearby information.
- The objective is to reach Goal (G) with the minimum total cost.

(b) Solution Approach (Uniform Cost Search)

1. Start from S.
2. Store nodes in a priority queue.
3. Expand the node with the lowest cost.
4. Skip obstacles.
5. Add 1 for each move and +2 for risk zones.
6. Repeat until the robot reaches G.

(c) AI Concepts

- **Intelligent Agent:** The robot senses the environment and makes decisions.
- **Rationality:** It always chooses the path with the lowest cost.
- **Environment Type:** Partially observable, dynamic, sequential, and discrete.

(d) Justification

Uniform Cost Search is suitable because it:

- Finds the minimum-cost path.
- Considers both movement and risk costs.

- Avoids obstacles.
- Produces the optimal solution.

Final Result**Optimal Path:**

$S \rightarrow (1,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (4,2) \rightarrow (4,3) \rightarrow G$

Total Cost = 8

Conclusion

The Uniform Cost Search (UCS) algorithm successfully guides the rescue robot to the survivor while minimizing the total travel cost and satisfying all the given constraints.